

Block 1 — Inspecting HTTP

Goal: see the request/response cycle from the lecture with your own eyes — on a real website, and on a page that *your own computer* serves.

Time: ~45 minutes · **You need:** Chrome and a terminal (Node from Block 0)

Open the dev tools in Chrome with **F12** (or right-click → *Inspect*) and switch to the **Network** tab. Make sure "**Disable cache**" is **not** ticked — we want to see caching in Task 4.

No course material on your computer yet, because Git didn't work in exercise 0? Read this sheet on GitHub. Tasks 1–3 need only Chrome.

Task 1 — Watch a real page load

1. Go to <https://developer.mozilla.org>.
2. With the **Network** tab open, **reload** the page (`Ctrl/Cmd + R`).
3. Answer:
 - Roughly how many requests did the page make? _____
 - Find the **first** row (the HTML document). Its **Method**: _____ — its **Status**: _____
 - Find one **image** request. Its **Type**: _____ — its **Size**: _____
 - What kinds of **Type** do you see besides `document` ? List three: _____

💡 The first row is the HTML. Every other row is a resource that HTML *asked for* — the exact "one page, many requests" idea from the lecture.

Task 2 — Read one request in detail

1. Click the **document** request (the first row).
2. Open the **Headers** panel and fill in:
 - Request **Method**: _____
 - Request **URL**: _____
 - Response **Status** code: _____
 - Response header **Content-Type** : _____
3. Open the **Response** panel. What do you see there? _____

Task 3 — Break things on purpose

1. In the address bar, take a real site and append a nonsense path, e.g.
`https://developer.mozilla.org/this-page-does-not-exist`.
2. With Network open, load it. What **status code** comes back? `_____` What does the first digit tell you about *who* is at fault? `_____`
3. Now type `http://github.com` (note: **http**, not https) and load it. Look at the **first** request's status — you should see a **3xx**.
 - Status code: `_____` — what does it mean? `_____`
 - What does the response `Location` header point to? `_____`

Task 4 — Be the server

In exercise 0, you served your first page. Now you watch what happens between the browser and your server.

1. If the server from exercise 0 still runs, stop it with `Ctrl + C`.
2. **Serve** the page in `starter/`:

```
cd exercises/week01/1/starter
npm install
npm start
```

Open the printed address (e.g. `http://localhost:3000`).

⚠ Serve it — don't double-click the file. `file://` isn't HTTP and won't show real status codes.

3. With the **Network** tab open, reload. You should see **four** requests. Match each to its **Type**:
`index.html` (`_____`), `styles.css` (`_____`), `logo.svg` (`_____`), `favicon.svg` (`_____`).
4. **Reload again**, and look at the **Status** column. Most files now return **304**. What does **304** tell the browser — and why does it save time? `_____`

Done when...

- [] You can name the method, status and `Content-Type` of a real page's document request
- [] You triggered a **404** and a **3xx** redirect, and know what each means
- [] Your own page runs at `http://localhost:3000`, and you saw its four requests
- [] You saw **304** on the second reload, and can explain it

At home

Do these tasks after class. They continue the numbering, so you can ask about them by number.

Task 5 — Map a request & response by hand

No browser for this one — write it out. A student's browser is about to request the course syllabus at `https://wd.hs-ansbach.de/syllabus`.

a) Write the **HTTP request** (request line + at least two headers):

```
_____ HTTP/1.1
_____ : _____
_____ : _____
```

b) Write a **plausible successful response** (status line + the header that tells the browser the body is an HTML page):

```
HTTP/1.1 _____
_____ : _____

<!DOCTYPE html> ...
```

c) The student mistypes the path as `/syllabus`. What status code do you expect, and which family (4xx/5xx) is it? _____

Stretch goals (optional)

- **HTTP is just text.** While your server from Task 4 runs, open a second terminal and run:

```
curl -v http://localhost:3000
```

Lines with `>` are the **request** curl sends; lines with `<` are the **response**. Find the request line, the status line and `Content-Type`. Compare them with your answers from Task 5. (Windows PowerShell: type `curl.exe` instead of `curl`.)

- Tick "**Disable cache**" and reload your page. What happens to the `304`s — and why?
- Click a request and open the **Timing** panel. Which phase takes the longest on a real site, and which on your own server?